

Eventim Disability Integration

*Barrierefreie Abwicklung von Ticketbestellungen &
Nachteilsausgleichsanträgen der Eventim-Webseite*

Version 1.0 – 29. Juli 2025

Eventim Disability Integration

Die Eventim Disability Integration stellt eine umfassende Lösung zur barrierefreien Abwicklung von Ticketbestellungen und Nachteilsausgleichsanträgen auf der Eventim-Plattform dar. Das System ermöglicht Menschen mit Behinderungen einen gleichberechtigten Zugang zu Veranstaltungen durch digitale Verifizierung von Schwerbehindertenausweisen und automatisierte Nachteilsausgleiche. Die Architektur basiert auf einem Next.js Frontend mit Express-Backend und PostgreSQL-Datenbank, wodurch eine skalierbare und benutzerfreundliche Plattform geschaffen wird, die sowohl gesetzliche Anforderungen erfüllt als auch den Komfort für die Zielgruppe deutlich erhöht. Dieses Dokument geht darauf ein, was das Ziel der Eventim Disability Integration ist, wie die Applikation aufgebaut ist und welche Funktionalitäten sie umsetzt.

Inhaltsverzeichnis

- [Einleitung](#)
 - [Projekthintergrund und Motivation](#)
- [Projektvision und Zielsetzung](#)
 - [Kernfunktionalitäten](#)
 - [Ziele](#)
 - [Zielgruppen und Stakeholder](#)
 - [Interne Stakeholder](#)
- [Setup](#)
 - [Installations-Guide](#)
 - [Login-Daten](#)
- [Architektur und eingesetzte Technologien](#)
 - [Ordnerstruktur](#)
 - [Frontend](#)
 - [Backend](#)
 - [Datenbank](#)
- [Verwendete Technologien](#)
 - [Frameworks und Libraries](#)
 - [Wiederverwendbare Codeabschnitte](#)
 - [Sessions](#)
- [Security und Fault Tolerance](#)
 - [Rollen und Zugriffsberechtigungen](#)
 - [Doppelte Validierung zwischen Front- und Backend bei CRUD-Operations](#)
 - [Ausfallsicherheit und Auto-Recovery](#)
- [Workflows](#)
 - [Registrierung von Nutzern](#)

- Ticketkauf
- Nachteilsausgleichsantrag stellen
- Nachteilsausgleichsantrag bearbeiten
- Nutzerrolle anpassen
- Event erstellen
- Tour löschen
- Benutzerkonto löschen
- Abmelden
- Profildaten aktualisieren
- Testkonzept
 - User-Tests
 - Backend-Tests
- Nächste Schritte
- Anhang
 - Datenbank-Modell
 - Frameworks

Einleitung

Projekthintergrund und Motivation

Die Eventim-Plattform ist eine der führenden Ticketing-Lösungen im deutschsprachigen Raum und verarbeitet jährlich Millionen von Ticket-Transaktionen für Konzerte, Festivals, Theater und Sportveranstaltungen. Trotz dieser marktführenden Position existiert eine bedeutende Lücke in der Barrierefreiheit und digitalen Inklusion für Menschen mit Behinderungen.

In Deutschland leben etwa 7,9 Millionen Menschen mit einer anerkannten Schwerbehinderung (Stand 2021). Diese Personengruppe hat nach dem Sozialgesetzbuch IX (SGB IX) und verschiedenen Landesgesetzen Anspruch auf Nachteilsausgleiche bei kulturellen Veranstaltungen. Die EU-Richtlinie zur Barrierefreiheit (European Accessibility Act) verpflichtet zudem bis 2025 zur digitalen Barrierefreiheit von Ticketing-Systemen.

Projektvision und Zielsetzung

Vision: "Eventim wird zur ersten vollständig inklusiven Ticketing-Plattform, die Menschen mit Behinderungen den gleichen komfortablen, digitalen Zugang zu kulturellen Erlebnissen ermöglicht wie allen anderen Nutzern."

Kernfunktionalitäten

- **Behindertenausweis-Verifizierung**
 - Zwei Einstiegspunkte
 - Registrierung eines neuen Benutzerkontos
 - Nachträgliche Beantragung über `/profile` möglich
 - Upload-Interface für Vorder- und Rückseite des Schwerbehindertenausweises
- **Automatisierte Nachteilsausgleiche**
 - Dynamische Darstellung von zusätzlichen Kategorien abhängig von Event und Venue, welche nur für diese Nutzergruppe buchbar sind und einen reduzierten Preis haben
 - Automatische Zubuchung eines kostenfreien Begleitpersonen-Tickets bei Buchung mit Merkzeichen "B" (Begleitperson) im Schwerbehindertenausweis

Ziele

Primäre Ziele:

- Vollständige Integration der Disability-Features in die bestehende Plattform
- Eliminierung manueller Prozesse durch intelligente Verifizierung
- Erfüllung aller rechtlichen Anforderungen Erschließung einer neuen Zielgruppe mit signifikantem Marktpotential

Sekundäre Ziele:

- Reduzierung der Support-Anfragen um 70%
- Steigerung der Kundenzufriedenheit in der Zielgruppe
- Positionierung als Accessibility-Leader im Ticketing-Markt

Zielgruppen und Stakeholder

Menschen mit Schwerbehinderung (Primary Users)

- Anzahl: ~7,9 Millionen in Deutschland
- Charakteristika: Diverse Behinderungsarten, unterschiedliche technische Affinität
- Bedürfnisse: Einfache, barrierefreie Buchung, automatische Vergünstigungen
- Pain Points: Aktuelle telefonische Buchung, Wartezeiten, Unsicherheit über Berechtigung

Begleitpersonen und Angehörige (Secondary Users)

- Anzahl: ~2–3 Millionen potentielle Nutzer
- Charakteristika: Oft technisch versierter, buchen stellvertretend
- Bedürfnisse: Transparente Abrechnung, klare Berechtigungsregeln

Interne Stakeholder

Service-Mitarbeiter

- Anzahl: ~50 Personen im Customer Service
- Bedürfnisse: Effiziente Tools zur Antragsbearbeitung, reduzierte Anruflast
- Erfolgsmetriken: 60% weniger Disability-bezogene Tickets

Admin-User (Content-Manager, Venue-Manager)

- Anzahl: ~20 Power-User
- Bedürfnisse: Einfache Konfiguration von barrierefreien Bereichen
- Erfolgsmetriken: Reduzierte Setup-Zeit für neue Events

Setup

Installations-Guide

1. **Repository klonen** und in das Projekt wechseln:

```
git clone <repo-url>
cd EventimDisabilityIntegration/eventim-disability-integration
```

2. **Abhängigkeiten installieren:**

```
npm install
```

3. **Datenbankzugang konfigurieren:**

Legen Sie im Verzeichnis `server` eine Datei `credentials.json` an. Beispiel:

```
{
  "host": "152.53.119.113",
  "port": 5433,
  "user": "postgres",
  "password": "example",
  "database": "db1",
  "sessionSecret": "example-secret"
}
```

4. **Entwicklungsumgebung starten:**

```
npm run dev
```

Damit starten sowohl das Next.js Frontend auf <http://localhost:3000>, das Backend unter <http://localhost:4000> sowie P2M zur Überwachung der Datenbank- und Backendverbindung.

Login-Daten

Im Zuge der Bewertung der Webseite bleibt es Ihnen offen, ob Sie sich selbst einen/mehrere eigene Accounts anlegen wollen oder ob sie bereits vordefinierte Accounts nutzen wollen. Folgende User wurden bereits gestellt und können genutzt werden:

Rolle	E-Mail	Passwort
USER	testUser1@test.de	testUser1@test.de
SERVICE	testService1@test.de	testService1@test.de
ADMIN	testAdmin1@test.de	testAdmin1@test.de

Architektur und eingesetzte Technologien

Die Anwendung besteht aus einem [Next.js](#) Frontend und einem [Express](#) Backend. Als Datenbank kommt [PostgreSQL](#) zum Einsatz. Die Wahl fiel auf diese Kombination, da sie leichtgewichtig, gut erweiterbar und auch ohne großen Konfigurationsaufwand lokal ausführbar ist. Next.js liefert die React basierte Oberfläche und kann sowohl statische Seiten als auch serverseitig gerenderte Inhalte bereitstellen. Express dient als schlanker REST-API Server, der über die `server`-Ordnerstruktur umgesetzt ist. Die Kommunikation zwischen Frontend und Backend erfolgt ausschließlich über JSON-basierte HTTP-Aufrufe.

Ordnerstruktur

Der gesamte Quellcode liegt im Ordner `eventim-disability-integration`. Die nachfolgende Tabelle bietet einen schnellen Überblick über alle relevanten Verzeichnisse:

Pfad	Inhalt
<code>src/pages/</code>	Sämtliche Next.js Seiten in <code>*.jsx</code> -Dateien. Unterordner wie <code>admin/</code> oder <code>artists/</code> bilden dynamische Routen ab und strukturieren den Code in für den Endnutzer intuitive Pfade
<code>src/components/</code>	Wiederverwendbare UI-Bausteine (Modals, Navigationsleisten, Karten usw.)
<code>src/hooks/</code>	Custom Hooks wie <code>useAuth</code> oder <code>useCart</code> , die zentrale Logik kapseln, welche auf mehreren Seiten/in mehreren Komponenten benötigt wird
<code>src/__tests__/</code>	Jest-Tests zur automatisierten Durchführung von Backend-Tests zur Sicherstellung der Verfügbarkeit aller REST-API-Endpunkte
<code>server/</code>	Express-Backend (<code>server.js</code>), DB-Anbindung (<code>db.js</code>), <code>backup_script.sql</code> und PM2-Konfiguration
<code>public/</code>	Statische Dateien, die unverändert von Next.js bereitgestellt werden (bspw. Icons, Logos usw.)

Frontend

Das Frontend einer Applikation ist die sichtbare Benutzeroberfläche (UI), über die Anwender mit den Funktionen und Daten der Anwendung arbeiten. Es ist aufgeteilt in Seiten (Pages) unter `/pages`, welche sich in Darstellung, Daten und Funktionalität unterscheiden. Nutzer können über das Frontend Abläufe der Applikationslogik starten, welche über das Backend und den Browser dynamisch Daten abrufen, sichern oder manipulieren.

Im Frontend der Applikation wurde Next.js in Kombination mit React eingesetzt, weil diese Lösung serverseitiges Rendering ermöglicht und eine klare Komponentenstruktur vorgibt. Alternative Umsetzungen wären etwa mit Vue.js/Nuxt oder Angular möglich gewesen, jedoch besitzt das Team bereits umfangreiche Erfahrung mit React, was die Wartung vereinfacht.

Seitenübersicht

Die Next.js Anwendung befindet sich unter `src/pages` und nutzt dynamische Routen. Wichtige Seiten sind:

Pfad	Zweck / Inhalte
<code>/</code>	Startseite mit Highlights sowie Künstler- und Tourübersicht
<code>/artists/[artist]</code>	Detailseite eines Künstlers und Übersicht zugehöriger aktiver Touren und deren Events
<code>/artists/[artist]/[tour]</code>	Übersicht aller Events einer ausgewählten Tour inkl. Verfügbarkeit von Tickets und Information über verfügbare barrierefreie Kategorien
<code>/artists/[artist]/[tour]/[event]</code>	Detailseite eines Events, verfügbarer Kategorien sowie die Möglichkeit, Tickets in den Warenkorb zu legen
<code>/registration</code>	Formular zur Kontoerstellung und Beantragung eines Nachteilsausgleichs mit Erstellung des Nutzerkontos
<code>/login</code>	Anmeldungsseite
<code>/profile</code>	Übersicht über gebuchte Events, getätigte Bestellungen, Verwaltung persönlicher Daten sowie einer FAQ-Section. Falls der Nutzer noch keinen Nachteilsausgleich beantragt hat kann dieser das hier tun oder den Status seines Antrags einsehen
<code>/checkout</code>	Mehrstufiger Bestellprozess (Versanddaten, Zahlung, Abschluss) zur Buchung von Tickets
<code>/admin</code>	Einstieg in alle Admin-Unterseiten zur Pflege von Künstlern, Ländern, Genres, Touren und Veranstaltungsorten
<code>/service</code>	Zugriffspunkt für Service-Mitarbeiter (u.a. Nachteilsausgleichsanträge und Account-Management) zur Verwaltung von Nachteilsausgleichsanträgen und Nutzerdaten zum Nachgang der telefonischen Servicetätigkeiten

Alle Seiten unter `/admin/*` und `/service/*` setzen entsprechende Berechtigungen voraus und sind nur Admin- bzw. Servicemitarbeitern gestattet. Falls ein Nutzer, welcher entweder nicht angemeldet ist oder nicht die notwendigen Berechtigungen besitzt auf diese Webseite geht, so wird dieser auf die Homepage zurückgewiesen.

Die Möglichkeit, einen Nachteilsausgleich als schwerbehinderte Person zu beantragen ist sowohl bei der Registrierung als auch nachträglich im Benutzerprofil möglich. Die Buchung von Tickets mit Nachteilsausgleich erfolgt über genannte Routen zur Buchung von Tickets und beinhaltet weitere Kategorien speozifisch für schwerbehinderte Nutzer, welche nur Nutzer buchen können mit validierten Schwerbehindertenstatus durch einen Service-Nutzer.

Im Folgenden wird das Backend der Applikation beschrieben.

Backend

Das Backend ist die serverseitige Schicht, die HTTP-Anfragen des Frontends entgegennimmt, über die Applikationslogik verarbeitet und anschließend auf die Datenbank zugreift, um Daten zu speichern oder abzurufen. Es stellt sogenannte Endpunkte zur Verfügung, mit denen die Applikationslogik über HTTP-Methoden (GET, POST, PUT, DELETE) angesprochen wird, um CRUD-Operationen auf den Daten auszuführen und diese in der Datenbank festzuhalten. Die Applikation nutzt einen lokal laufenden Server, der über das Skript `server/server.js` gestartet wird und unter `http://localhost:4000/` erreichbar ist.

Im Backend der Applikation wurde Express als leichtgewichtiges Framework eingesetzt, da es eine minimalistische Struktur besitzt und Middleware sehr flexibel eingebunden werden kann. Im Gegensatz zu komplexeren Lösungen wie NestJS ermöglicht Express einen schnellen Einstieg und volle Kontrolle über den Request-Flow. Alternative Umsetzungen wären mit NestJS oder Fastify möglich gewesen, doch Express ist in der Node.js-Community weit verbreitet und dementsprechend ausgezeichnet dokumentiert.

REST-Endpunkte

Im Folgenden sind die wichtigsten Routen des Express-Servers (siehe `server/server.js`) aufgeführt:

Methode	Pfad	Beschreibung
POST	<code>/register-user</code>	Registrierung eines neuen Nutzers inkl. optionaler Behindertenausweis-Daten
POST	<code>/login-user</code>	Benutzeranmeldung, legt Session-Cookie an
GET	<code>/session-status</code>	Prüft, ob ein Nutzer eingeloggt ist und liefert Profilinformationen
POST	<code>/logout</code>	Beendet die aktuelle Session
GET	<code>/user-address</code>	Liefert die hinterlegte Anschrift des eingeloggten Nutzers
GET	<code>/disability-marks</code>	Auflistung möglicher Markierungen des Behindertenausweises
GET	<code>/pending-disability-requests</code>	Offene Anträge auf Nachteilsausgleich für den Service-Bereich

Methode	Pfad	Beschreibung
POST	/disability-requests/:id/accept	Antrag eines Nutzers akzeptieren
POST	/disability-requests/:id/decline	Antrag eines Nutzers ablehnen
GET	/artists	Auflistung aller Künstler
POST	/create-artist	Neuen Künstler anlegen
GET	/tours-detailed	Touren inkl. Events und Zugänglichkeitsdaten
POST	/create-tour	Neue Tour anlegen
GET	/venues-detailed	Liste aller Veranstaltungsorte mit Areas
POST	/create-venue	Neuen Veranstaltungsort anlegen
POST	/cart-items	Ticket zur Warenkorb-Session hinzufügen
GET	/checkout-items	Aktuellen Checkout laden
POST	/orders	Bestellung aus abgeschlossenem Checkout erzeugen

Aus Platzgründen wurde auf die Darstellung aller Backend-Routen verzichtet. Weitere Endpunkte finden sich direkt im Quellcode von `server/server.js`.

Im Folgenden wird die verwendete Datenbank sowie deren Tabellen betrachtet.

Datenbank

Eine Datenbank ist eine Anwendung, die Daten anhand eines vordefinierten Schemas organisiert, dauerhaft ablegt und über standardisierte Schnittstellen abfragbar macht. Realisiert werden diese Abfragen mittels einer Library

Die eingesetzte Datenbank basiert auf PostgreSQL und wurde einmalig über `server/backup_script.sql` erstellt. Enthalten sind Tabellen für Benutzer, Rollen, Künstler, Touren, Events, Veranstaltungsorte sowie Tabellen zur Abwicklung von Bestellungen und zur Speicherung von Disability-Merkmalen. Hierbei wurde darauf geachtet, dass alle Metadaten ebenfalls in der Datenbank angepasst werden können, somit flexibel angepasst und erweitert werden können.

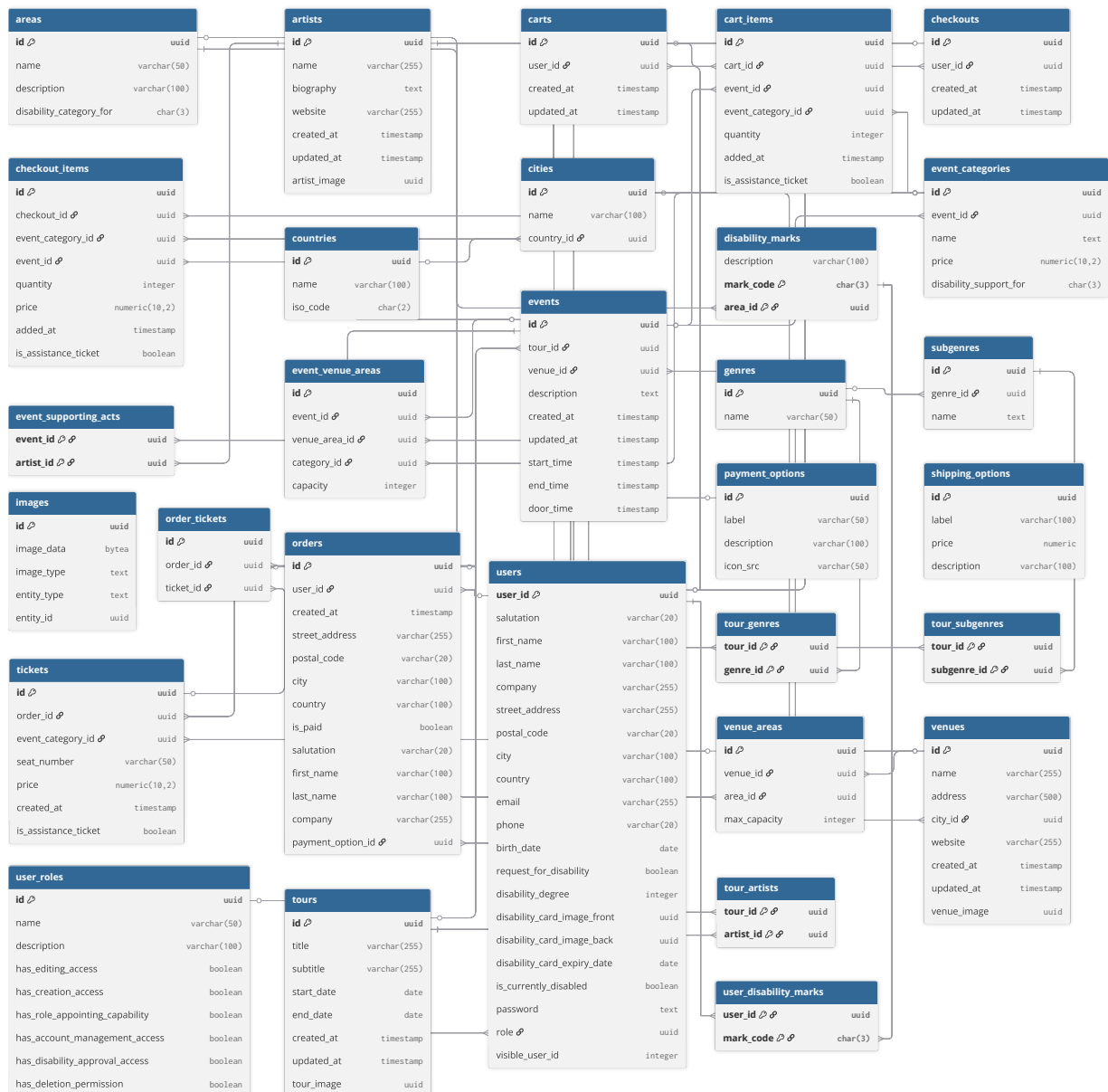
Es wurde sich für eine PostgreSQL Datenbank entschieden, da diese ACID-konforme Transaktionen sowie eine ausgereifte Query-Engine bietet und gleichzeitig JSON-Datenstrukturen unterstützt. Alternativen wie MySQL oder MongoDB wurden geprüft, jedoch erschien PostgreSQL aufgrund der breiten Community-Unterstützung und der stabilen Erweiterbarkeit am sinnvollsten.

Über das `pg`-Modul von PostgreSQL wird zwischen Backend und Datenbank ein Connection-Pool aufgebaut, welcher eine effiziente Verwaltung von Datenbankverbindungen ermöglicht. Diese Technik reduziert den Overhead beim Öffnen und Schließen von Verbindungen erheblich und verbessert dadurch die Performance des Systems. Der Pool wird beim Start des Servers initialisiert und bleibt während der gesamten Laufzeit bestehen.

Die Kommunikation erfolgt über parametrisierte SQL-Queries, die vor SQL-Injection-Angriffen schützen. Sollte die Datenbankverbindung unterbrochen werden, versucht der in `server/db.js` implementierte Circuit-Breaker automatisch, den Pool neu zu initialisieren. Diese Architektur gewährleistet eine robuste und ausfallsichere Datenpersistenz für alle Transaktionen der Anwendung.

Im Folgenden ein Überblick über die wichtigsten Datenbanktabellen und ihre Beziehungen zueinander.

Datenbanktabellen



Zu sehen sind alle verwendeten Datenbanktabellen sowie ihre Beziehungen zueinander. Die Datenbank liegt normiert in 3. Normalform vor, um Redundanzen in Daten und Abhängigkeiten sowie Anomalien vorzubeugen. Zur Sicherstellung von Datenintegrität kann jedes Element jeder Tabelle mittels seiner eindeutigen V4-UUID identifiziert werden. Fremdschlüsselbeziehungen

Für einfache Lookups existieren diverse Join-Tabellen (z. B. `tour_genres`). Bei der Konstruktion des Datenbankmodells wurde stets darauf geachtet, dass die Datenbank in dritter Normalform

vorliegt, somit Anomalien durch Löschen oder Anpassungen von Daten weitestgehend midigiert werden können.

Eine detaillierte Ansicht aller Tabellen, eine zugehörige Beschreibung sowie ihre Zusammenhänge zueinander befindet sich im Anhang.

Im Folgenden werden weitere Technologien beschrieben, welche als Runtime / Framework / Library in den Code der Applikation eingebunden wurden.

Verwendete Technologien

Frameworks und Libraries

Neben den eingesetzten Frameworks wurden in diesem Projekt mehrere Libraries eingebunden, welche im Folgenden aufgelistet werden.

Komponente/Bibliothek	Kategorie	Zweck
Node.js	Runtime	Ausführung von Backend und Next.js
Next.js	Framework	React-basiertes Frontend mit Server Side Rendering
React	Library	UI-Komponenten im Browser
Express	Backend Framework	REST-API und Routing
PostgreSQL	Datenbank	Persistente Speicherung aller Daten
pg	Library	Zugriff auf PostgreSQL im Backend
bcrypt	Library	Hashing von Passwörtern
multer	Library	Verarbeiten von Datei-Uploads (multipart/form-data) im Server zur Speicherung in der DB
react-router-dom	Library	Clientseitige Navigation
react-toastify	Library	Anzeigen von Toast-Benachrichtigungen
opossum	Library	Circuit-Breaker für Fehlerbehandlung und automatisierte Neustarts
PM2	Tool	Prozessmanager für den Server zur Absicherung der Applikation gegenüber Störungen
nodemon	Tool	Automatischer Neustart im Entwicklungsmodus
uuid	Library	Erzeugen eindeutiger V4 UUIDs
Testing Library / Jest	Testframeworks	Unit- und Integrationstests für Frontend (und bei Bedarf Backend)

Es wurde insbesondere `react-toastify` genutzt, um auf der Seite ein konsistentes Benachrichtigungssystem umzusetzen. Darüber hinaus sorgen `opossum` und `PM2` für eine resiliente Fehlerbehandlung im Backend und einen stabilen Betrieb.

Wiederverwendbare Codeabschnitte

Eine Komponente im Kontext dieser Arbeit ist ein kapselbarer UI-Baustein auf React-Basis, wohingegen ein Hook wiederverwendbare Logik wie Zustandsverwaltung oder Seiteneffekte

abbildet. Beide Konzepte eignen sich dazu, komplexe Abläufe zu abstrahieren und die Wiederverwendbarkeit des Codes signifikant zu erhöhen.

Der Quellcode unter `src` gliedert sich in wiederverwendbare React-Komponenten und mehrere Custom Hooks, welche auf mehreren Seiten implementiert wurden

Komponenten

Datei	Einsatz
<code>nav-bar.jsx</code>	Hauptnavigation mit Suche und Warenkorb-Anzeige
<code>footer.jsx</code>	Gemeinsamer Seitenfuß
<code>LoadingOverlay.jsx</code>	Überblendet die Seite während Daten geladen werden
<code>DisabilityRequestModal.jsx</code>	Dialog zur Beantragung eines Nachteilsausgleichs
<code>DeleteAccountModal.jsx</code>	Bestätigungsdialog zum Löschen des Accounts
<code>smallArtistCard.jsx</code> <code>smallTourCard.jsx</code>	/ Vorschaukarten auf der Startseite

Die Komponenten sind so aufgebaut, dass sie in verschiedenen Seiten wiederverwendet werden können. Jede Komponente wurde nach dem Prinzip der Single Responsibility entwickelt und nutzt Props für die Datenweitergabe. Beispielsweise rendert `smallArtistCard.jsx` einen Künstler in verschiedenen Kontexten (Startseite, Suche, Verwaltung) mit konsistenter Darstellung aber unterschiedlichen Callback-Funktionen. Die Komponenten implementieren zudem das React-Memoization-Pattern, wodurch unnötige Re-Renderings vermieden und die Performance optimiert wird.

Hooks

Hook	Zweck
<code>useAuth</code>	Kümmert sich um Login-Status und hält Userdaten clientseitig vor
<code>useCart</code>	Verwaltet den Warenkorb und lädt Items vom Server
<code>useRequireAccess</code>	Leitet unberechtigte Nutzer auf die Login-Seite um
<code>useValidation</code>	Hilft bei Formularvalidierungen

Die Hooks kapseln wiederverwendbare Geschäftslogik und stellen eine konsistente Schnittstelle zum Backend bereit. Während `useAuth` Session-Management und Berechtigungsprüfung implementiert, orchestriert `useCart` die asynchrone Kommunikation mit dem Server für Warenkorb-Operationen. Diese modulare Struktur reduziert Redundanz und garantiert einheitliches Verhalten über die gesamte Anwendung hinweg.

Sessions

Eine Session ist eine temporäre Verbindung zwischen Client und Server, die Benutzerdaten während der Interaktion mit einer Webanwendung speichert. Die Anwendung nutzt serverseitiges Session-Management mittels `express-session`. Die Session-Verwaltung basiert auf einem Cookie-basierten Ansatz, bei dem der Browser nur eine einzigartige Session-ID enthält, während die eigentlichen Session-Daten sicher auf dem Server verbleiben.

Session-Architektur

- **Session-ID:** Wird als Cookie namens `sid` im Browser gespeichert
- **Session-Store:** Standardmäßig nutzt die Anwendung einen `MemoryStore` im Express-Server
- **Lebensdauer:** Sessions sind auf ihre Lebensdauer begrenzt, verlängern sich jedoch bei Aktivität
- **Sicherheit:** Die Cookies werden mit dem Flag `httpOnly` gesendet, um Client-seitigen JavaScript-Zugriff zu verhindern

Gespeicherte Daten

In den Sessions werden abhängig vom Anwendungsfall verschiedene Informationen gespeichert:

Session-Typ	Gespeicherte Daten	Lebensdauer
Login-Session	<code>userId</code> , <code>email</code> , Rollenrechte	60 Minuten
Checkout-Session	Warenkorb-Items, Versand- und Zahlungsdaten	15 Minuten Inaktivität

Session-Workflow

1. Bei Anmeldung erstellt der Server eine neue Session und füllt sie mit Benutzerdaten
2. Während der Navigation werden Session-Daten genutzt, um Berechtigungen zu prüfen
3. Im Kaufprozess werden Artikel und Checkout-Informationen in der Session gespeichert
4. Bei Abmeldung oder Timeout wird die Session zerstört

Die Session-Daten werden serverseitig validiert, um unbefugte Zugriffe oder Manipulationen zu verhindern. Dies ist besonders wichtig für die Rollenverwaltung und Berechtigungsprüfungen beim Zugriff auf geschützte Bereiche der Anwendung.

Security und Fault Tolerance

Rollen und Zugriffsberechtigungen

Die Tabelle `user_roles` definiert Berechtigungen. Aktuell existieren drei Rollen:

Rollenname	Beschreibung	Edit Entities	Create Entities	Appoint Roles	Delete Entities	Disability Approval	Account Mgmt
user	Regulärer Eventim Nutzer	false	false	false	false	false	false
service	Service-Mitarbeiter von Eventim	false	false	false	false	true	true
admin	Admin-Nutzer (alle Rechte)	true	true	true	true	true	true

Damit wird garantiert, dass nur Nutzer mit Berechtigungen ausgewählte Aktionen durchführen dürfen und Rechte durch einen Administrator verwaltet werden können. Hierbei wurde zusätzlich darauf geachtet, dass im System mindestens ein Administrator bestehen muss, indem diesem das Löschen seines eigenen Accounts verweigert wird, sofern kein anderer Admin existiert.

Zusätzlich ist es Nutzern mit fehlenden Berechtigungen nicht möglich, auf Seiten zu navigieren, zu denen sie keine Berechtigung haben, indem in der Login-Session die Rollenrechte des Nutzers gespeichert werden und auf ausgewählten Seiten abgefragt wird, ob diese erfüllt sind oder nicht. Sind diese nicht berechtigt, die Seite aufzurufen, so werden sie mittels eines `307: Temporary Redirect` auf die Ursprungsseite zurückverwiesen.

Doppelte Validierung zwischen Front- und Backend bei CRUD-Operations

Um im Falle eines Datenbank- oder Backend-Fehlers dem Nutzer weiterhin ein funktionsfähiges System zu bieten, werden Fehlerzustände serverseitig abgefangen und dem Frontend in strukturierter Form gemeldet. Beim Versuch, eine Tour bzw. ein Event zu löschen validiert zunächst das Frontend, ob zu dieser Tour/ einem Event dieser Tour bereits Tickets existieren. Nur wenn keine Tickets in `orders` hinterlegt sind, erscheint die Möglichkeit das Event zu löschen.

Sollte das Frontend in einen Fehler laufen und dennoch ein "Löschen"-Icon anzeigen wirft das Backend in `server/server.js` einen Fehler, dass diese Aktion nicht erlaubt ist, bevor dads Event auf der Datenbank gelöscht wird. Dieser wird vom Frontend entgegengenommen und in Form einer Fehlermeldung dem User angezeigt, dass die von ihm durchgeführte Aktion nicht möglich ist.

Folgende Fälle werden durch die Applikation abgedeckt:

Komponente	HTTP-Operation	Verhalten
Admin-Bereich	<code>GET</code>	Weiterleitung zur Login-Seite bei fehlenden Rechten
Stadt	<code>DELETE</code>	Popup: Stadt in Stadion genutzt

Komponente	HTTP-Operation	Verhalten
Venue	DELETE	Popup: Tickets vorhanden, Löschen nicht möglich
Land	DELETE	Popup: Städte in Stadion genutzt
Nutzerkonto	DELETE	Fehler: bestehende Buchungen verhindern Löschung
Event	DELETE	Löschen nicht möglich bei verkauften Tickets
Tour	DELETE	Löschen nicht möglich bei gebuchten Events
Artist	DELETE	Künstler nicht löschar
Genre	DELETE	Popup: Genre in Tour genutzt
Sub-Genre	DELETE	Popup: Sub-Genre in Tour genutzt
Ticketkauf	PUT	Button ausgegraut bei Maximalanzahl
SB-Antrag	DELETE	Antrag wird entfernt
Event-Sitzplatzkonf.	POST	Fehler: Kategorie-Wert muss > 0 sein
SB-Ticketkauf	POST	Button ausgegraut, Kauf blockiert

Umgesetzt wird dies durch ein Zusammenspiel aus clientseitigen Guards und serverseitiger Validierung, welche jede Manipulation der Daten prüft. Diese doppelte Validierung erlaubt es, dass die Daten trotz Fehler in Front- und Backend nicht fehlerhaft auf der Datenbank gesichert/manipuliert werden.

Ausfallsicherheit und Auto-Recovery

Im Falle eines Absturz des Backends oder der Datenbank wird dieses durch die Verwendung von **PM2** und **opossum** automatisch neugestartet. Übersteigt die Antwortzeit einer Datenbank-Query den in **opossum** gesetzten Timeout, werden Requests an die Datenbank blockiert. **PM2** startet dann einen neuen Worker, der Timer wird zurückgesetzt und ein neuer Connection-Pool zwischen Datenbank und Backend aufgebaut. Die Ausfallsicherheit wird durch folgende Maßnahmen gewährleistet:

- **server/db.js** implementiert einen Circuit-Breaker auf Basis von **opossum**, der bei Datenbankfehlern nach 5 Sekunden einen Reconnect versucht und verhindert, dass wiederholte Fehler das System überlasten
- **server/server.js** startet erst, wenn eine stabile DB-Verbindung besteht, und führt alle 15 Minuten Aufräumbjobs für abgelaufene Sessions durch
- Der Express-Server läuft unter **PM2** im Cluster-Modus, wodurch bei Abstürzen ein Neustart erfolgt
- Periodische Health-Checks alle 10s prüfen die Verbindung zur Datenbank und lösen bei Bedarf einen kontrollierten Neustart aus
- Ungefangene Exceptions werden zentral protokolliert und führen zu einem geordneten Exit-Code, damit PM2 sofort einen neuen Prozess starten kann

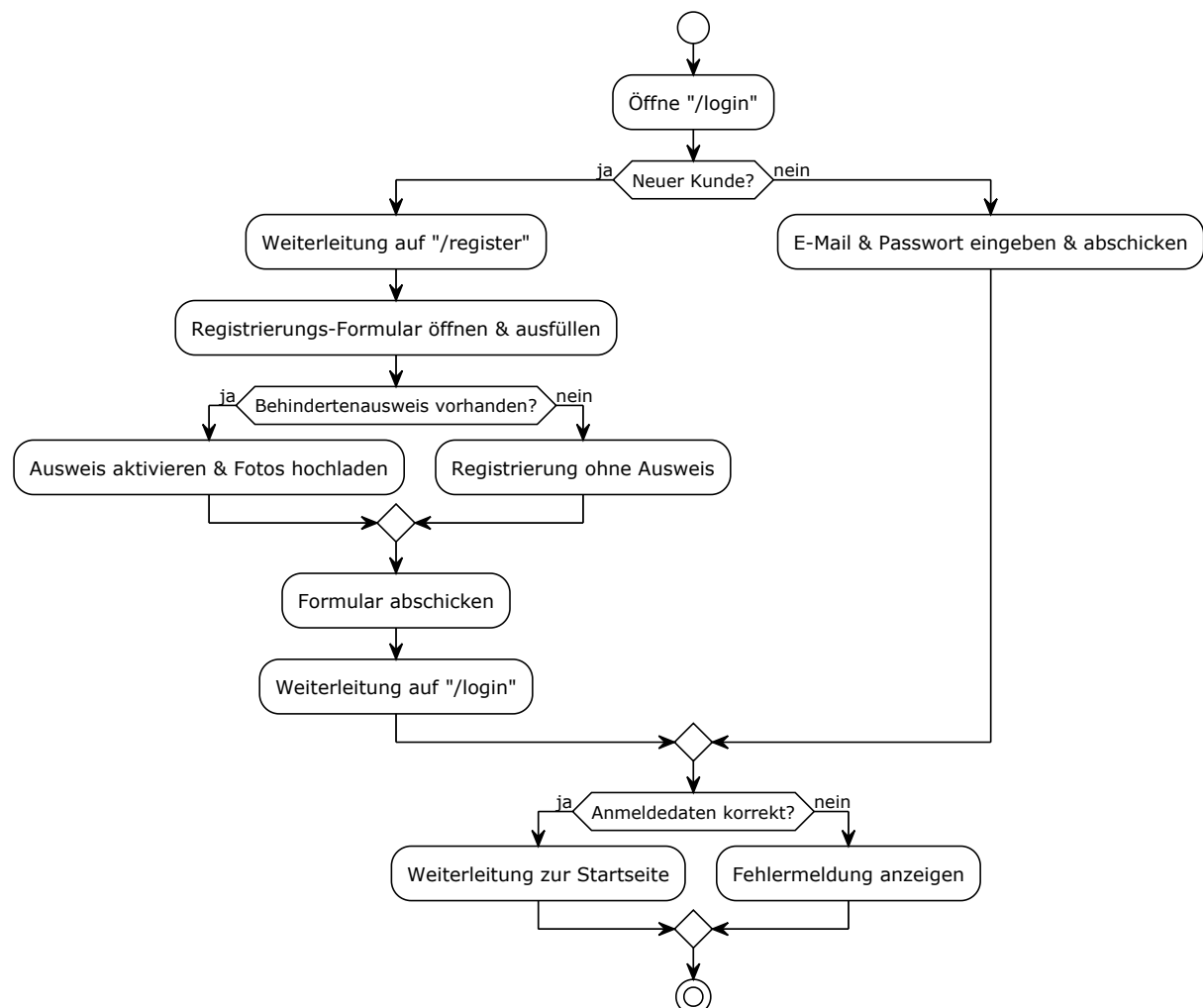
Diese Architektur gewährleistet, dass temporäre Datenbankausfälle die Anwendungsverfügbarkeit nur minimal beeinträchtigen und das System selbstständig neustartet.

Workflows

Die folgenden Aktivitätsdiagramme visualisieren typische Abläufe im System. Sie zeigen jeweils, auf welchen Seiten sich der Nutzer befindet und welche Daten einzugeben sind.

Registrierung von Nutzern

Registration Process



Überblick

Der "Registrierungsprozess" beschreibt den Fluss der Benutzerregistrierung im System. Er bietet sowohl eine Option für neue Kunden zur vollständigen Registrierung als auch einen direkten Anmeldepfad für bestehende Kunden. Ein zentraler Aspekt ist die optionale Verifizierung mittels Behindertenausweis.

Ablauf

1. Startpunkt: Öffnen der Login-Seite

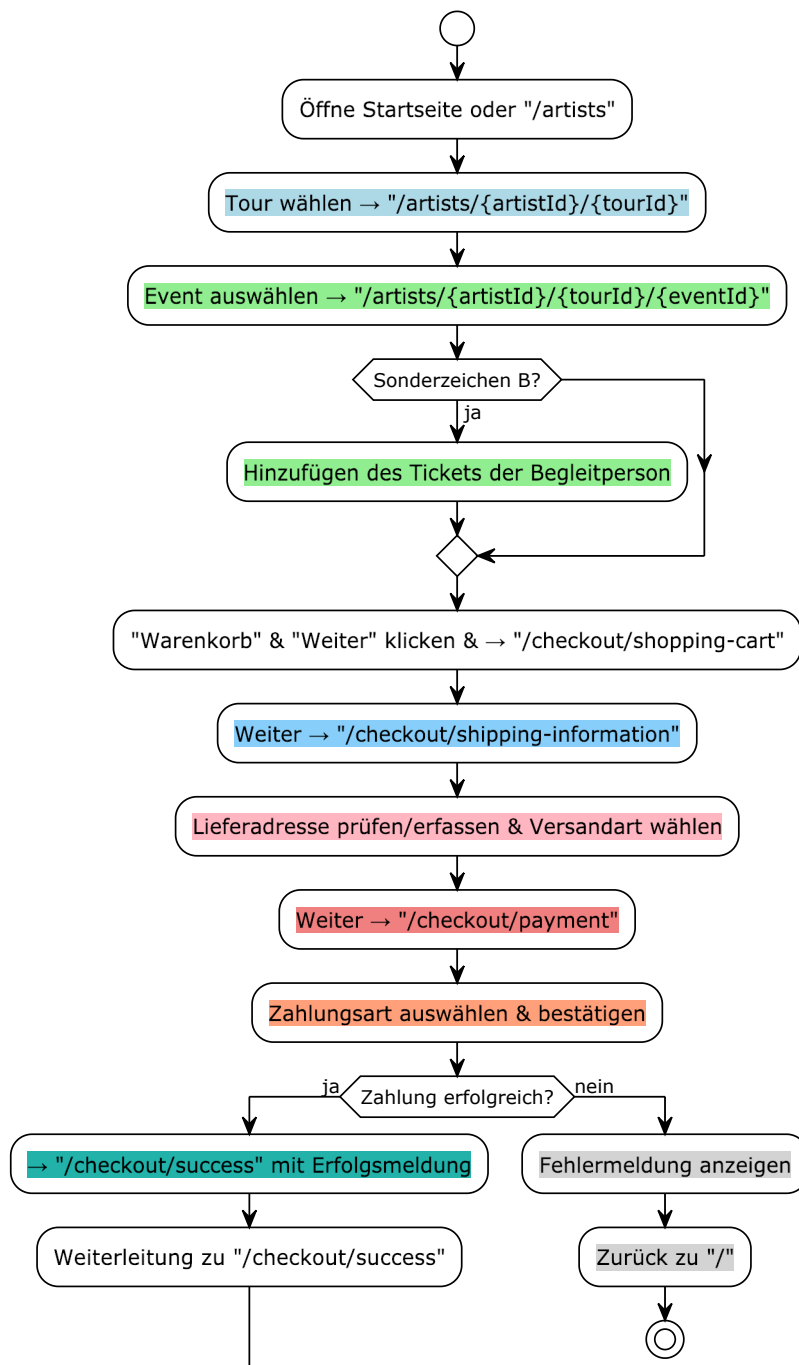
- Der Prozess beginnt mit dem Aufrufen der /login-Seite.

2. Entscheidung: Neuer Kunde?

- Wenn "Ja" (Neuer Kunde):
 - Weiterleitung auf "/register": Der Benutzer wird zur Registrierungsseite weitergeleitet.
 - Registrierungs-Formular öffnen & ausfüllen: Der Benutzer füllt das Registrierungsformular aus.
 - Entscheidung: Behindertenausweis vorhanden?
 - Wenn "Ja":
 - Ausweis aktivieren & Fotos hochladen: Der Benutzer aktiviert die Ausweisfunktion und lädt entsprechende Fotos hoch.
 - Wenn "Nein":
 - Registrierung ohne Ausweis: Die Registrierung wird ohne Ausweisverifizierung fortgesetzt.
 - Formular abschicken: Das ausgefüllte Formular wird abgeschickt.
 - Weiterleitung auf "/login": Nach dem Abschicken wird der Benutzer zurück zur Login-Seite geleitet.
- Wenn "Nein" (Bestehender Kunde):
 - E-Mail & Passwort eingeben & abschicken: Der Benutzer gibt direkt seine Anmeldedaten auf der Login-Seite ein und sendet sie ab.

Ticketkauf

Kaufprozess



Überblick

Der Kaufprozess bildet den vollständigen Ablauf zur Ticketbuchung auf der Plattform ab – von der Tourauswahl über den Checkout bis zur Zahlungsbestätigung. Dabei werden Nutzer schrittweise durch Auswahlmöglichkeiten, Eingabemasken und Zahlungsoptionen geführt, um eine reibungslose Bestellung zu ermöglichen.

Tour- und Eventauswahl

1. Einstieg über die Startseite oder Künstlerübersicht (/artists)

2. Auswahl einer bestimmten Tour eines Künstlers
3. Auswahl eines konkreten Events innerhalb der Tour
4. Optionales Hinzufügen eines Tickets für eine Begleitperson bei aktivem Sondermerkmal (z. B. Sonderzeichen B)

Checkout-Prozess

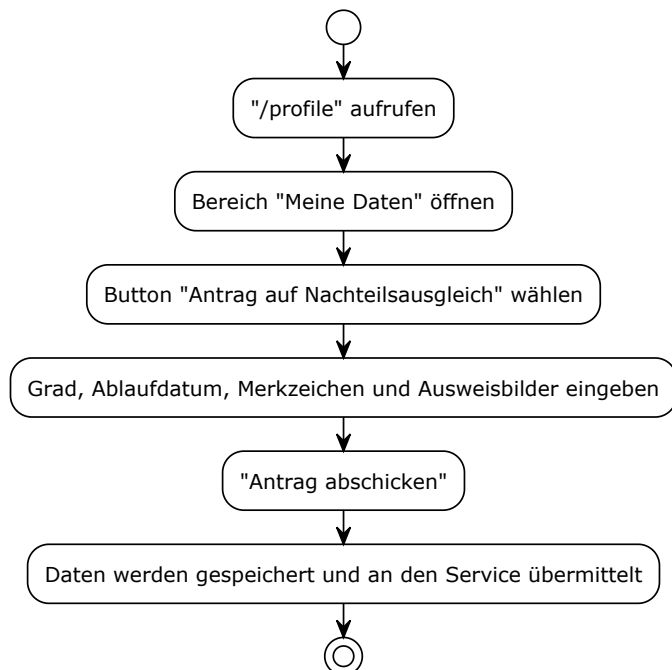
1. Hinzufügen der Eventtickets in den Warenkorb
2. Weiterleitung zur Seite für Versandinformationen
3. Eingabe oder Prüfung der Lieferadresse und Auswahl der Versandart
4. Fortsetzung zum Zahlungsbereich

Zahlungsabwicklung

1. Auswahl und Bestätigung der gewünschten Zahlungsart
2. Erfolgreiche Zahlung führt zur Weiterleitung auf eine Bestätigungsseite mit Erfolgsnachricht
3. Bei fehlgeschlagener Zahlung: Anzeige einer Fehlermeldung und Rückleitung zur Startseite

Nachteilsausgleichsantrag stellen

Antrag auf Nachteilsausgleich



Überblick

Die Funktion "Antrag auf Nachteilsausgleich" ermöglicht es Benutzern, einen Antrag zur Berücksichtigung besonderer Bedürfnisse einzureichen. Dies beinhaltet die Angabe relevanter Daten wie Grad des Nachteilsausgleichs, Ablaufdatum und das Hochladen von Ausweisbildern zur Übermittlung an den Service.

Bearbeitungsablauf

1. Profilbereich aufrufen:

- Der Prozess beginnt mit dem Aufrufen des persönlichen Profilbereichs unter dem Pfad `"/profile"`.

2. Bereich "Meine Daten" öffnen:

- Innerhalb des Profilbereichs öffnet der Benutzer den Abschnitt "Meine Daten".

3. "Antrag auf Nachteilsausgleich" wählen:

- Der Benutzer klickt auf den Button oder die Option "Antrag auf Nachteilsausgleich".

4. Daten eingeben:

- Es öffnet sich ein Formular, in dem der Benutzer den Grad des Nachteilsausgleichs, das Ablaufdatum, relevante Merkzeichen und Ausweisbilder eingibt.

5. Antrag abschicken:

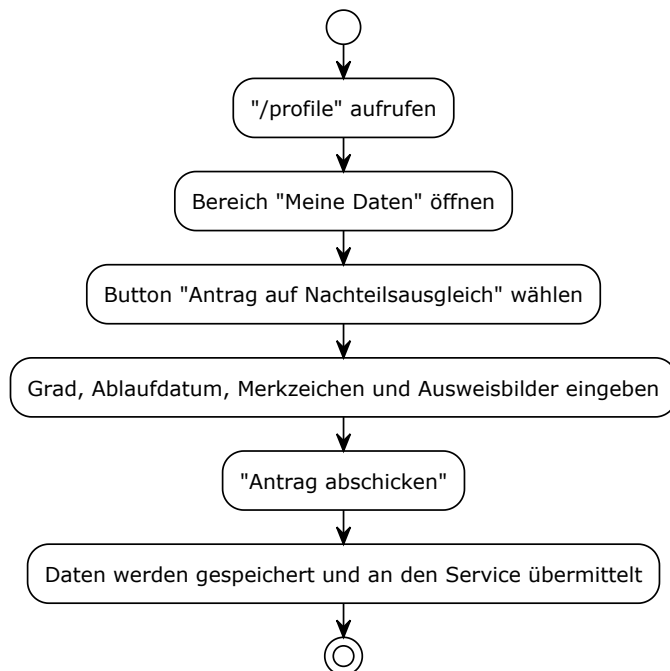
- Nachdem alle erforderlichen Informationen eingegeben wurden, klickt der Benutzer auf die Schaltfläche "Antrag abschicken".

Systemverarbeitung

- Daten werden gespeichert und an den Service übermittelt:
- Das System speichert die eingegebenen Daten und leitet den Antrag zur weiteren Bearbeitung an den zuständigen Service weiter.

Nachteilsausgleichsantrag bearbeiten

Antrag auf Nachteilsausgleich



Überblick

Dieser Prozess unterstützt autorisierte Mitarbeitende bei der strukturierten Prüfung und Bearbeitung von Anträgen auf Nachteilsausgleich. Die Entscheidung basiert auf der Sichtung hochgeladener Daten und Ausweisfotos, wobei die Bearbeitung direkt im System erfolgt.

Bearbeitungsablauf

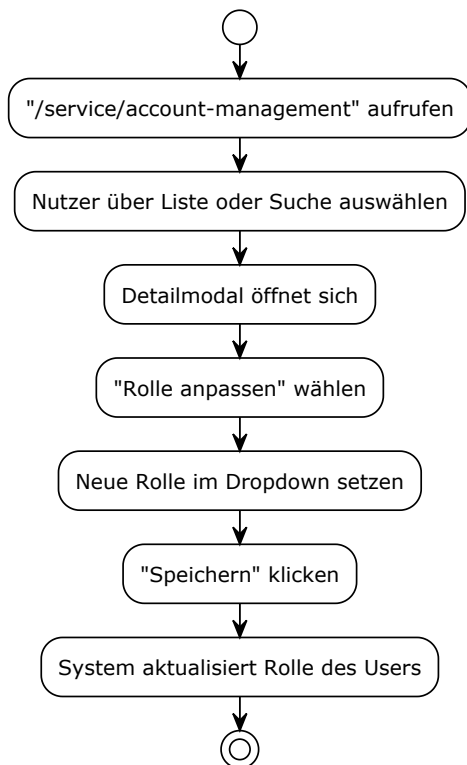
1. Öffnen der Antragsübersicht über den Pfad `"/service/compensation-for-disadvantages-requests"`
2. Durchsuchen der Tabelle „Offene Anträge“
3. Anklicken eines Eintrags, wodurch ein Detailmodal mit allen relevanten Angaben erscheint
4. Sichtprüfung der Antragsdaten und der hochgeladenen Ausweisdokumente

Entscheidung und Ergebnis

1. Ist der Antrag formal korrekt:
 - 1.1. Klick auf „Annehmen“, der Antrag wird genehmigt
2. Ist der Antrag fehlerhaft oder unvollständig:
 - 2.1. Klick auf „Ablehnen“, der Antrag wird zurückgewiesen
3. Nach der Entscheidung wird die Antragsliste automatisch aktualisiert und der bearbeitete Eintrag entfernt

Nutzerrolle anpassen

Benutzerrolle ändern



Überblick

Die Funktion „Benutzerrolle ändern“ ermöglicht es autorisierten Administratoren, die Rollen einzelner Nutzer im System anzupassen. Der Prozess erfolgt über eine zentrale Verwaltungsoberfläche und wird durch ein Modal-Dialogsystem unterstützt.

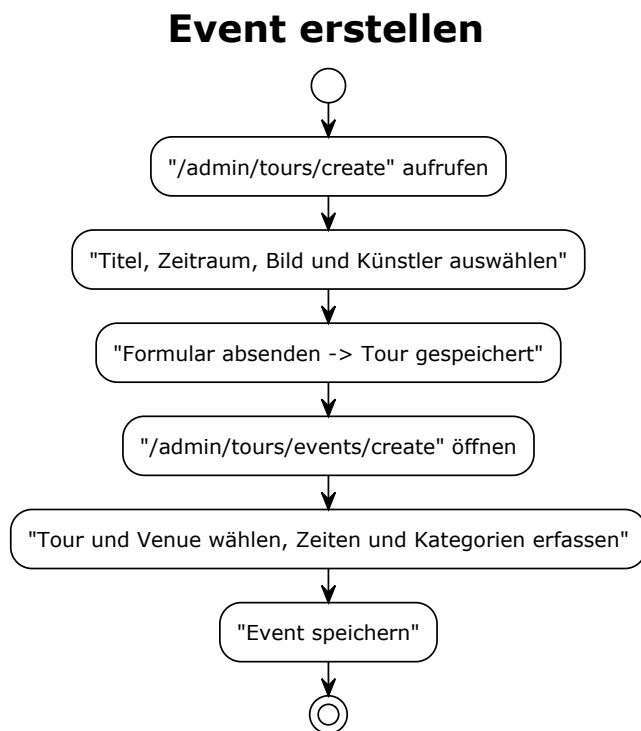
Anpassung der Rolle

1. Aufrufen des Benutzerverwaltungsbereichs unter `"/service/account-management"`
2. Auswahl eines Nutzers über die Ergebnisliste oder durch Nutzung der Suchfunktion
3. Nach Auswahl eines Nutzers öffnet sich automatisch ein Detailmodal
4. Im Modal wird die Option „Rolle anpassen“ ausgewählt
5. Eine neue Rolle wird im Dropdown-Menü festgelegt
6. Die Änderung wird durch Klick auf „Speichern“ bestätigt
7. Das System aktualisiert die Benutzerrolle unmittelbar nach der Aktion

Hinweise

- Nur autorisierte Rollen (z. B. Admin) dürfen Änderungen an Benutzerrollen vornehmen
- Die Änderungen wirken sich unmittelbar auf Berechtigungen und Zugriffsrechte des Nutzers aus

Event erstellen



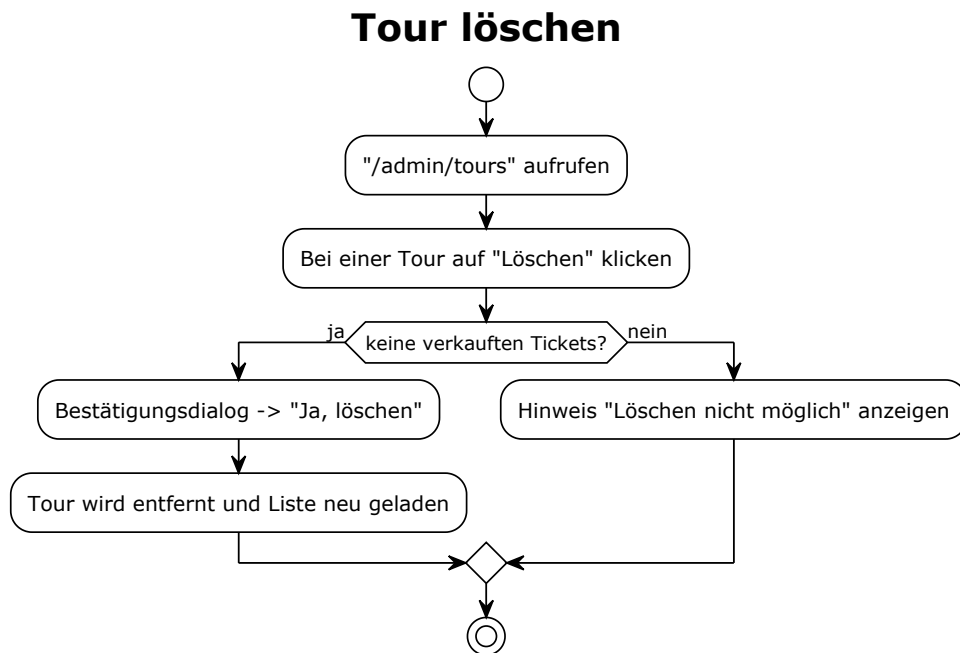
Überblick

Diese Funktion ermöglicht Administratoren die Erstellung neuer Events im System. Der Ablauf besteht aus zwei Stufen: Zuerst wird eine übergeordnete Tour mit Metadaten angelegt, anschließend können zugehörige Einzel-Events hinzugefügt und konfiguriert werden.

Ablauf

1. Aufruf des Erstellungsformulars über den Pfad "/admin/tours/create"
2. Eingabe grundlegender Tourdaten:
 - 2.1. Titel
 - 2.2. Zeitraum
 - 2.3. Bildmaterial
 - 2.4. Künstlerzuordnung
3. Absenden des Formulars, wodurch die neue Tour gespeichert wird
4. Im nächsten Schritt:
 - 4.1. Öffnen der Event-Erstellung unter "/admin/tours/events/create"
 - 4.2. Auswahl von Tour und Veranstaltungsort (Venue)
 - 4.3. Erfassung von Event-spezifischen Informationen:
 - 4.3.1 Datum & Uhrzeit
 - 4.3.2 Kategorien
5. Abschließend wird das Event gespeichert und dem System hinzugefügt

Tour löschen



Überblick

Die Funktion „Tour löschen“ erlaubt es Administratoren, nicht mehr benötigte Touren aus dem System zu entfernen. Vor der Löschung wird sichergestellt, dass keine Tickets für die betreffende Tour verkauft wurden, um eine versehentliche Datenlöschung zu verhindern.

Ablauf

1. Aufruf des Tour-Verwaltungsbereichs über "/admin/tours"
2. Auswahl einer Tour in der Übersicht

3. Klick auf die Option „Löschen“
4. Das System prüft, ob die Tour gelöscht werden kann

Entscheidung

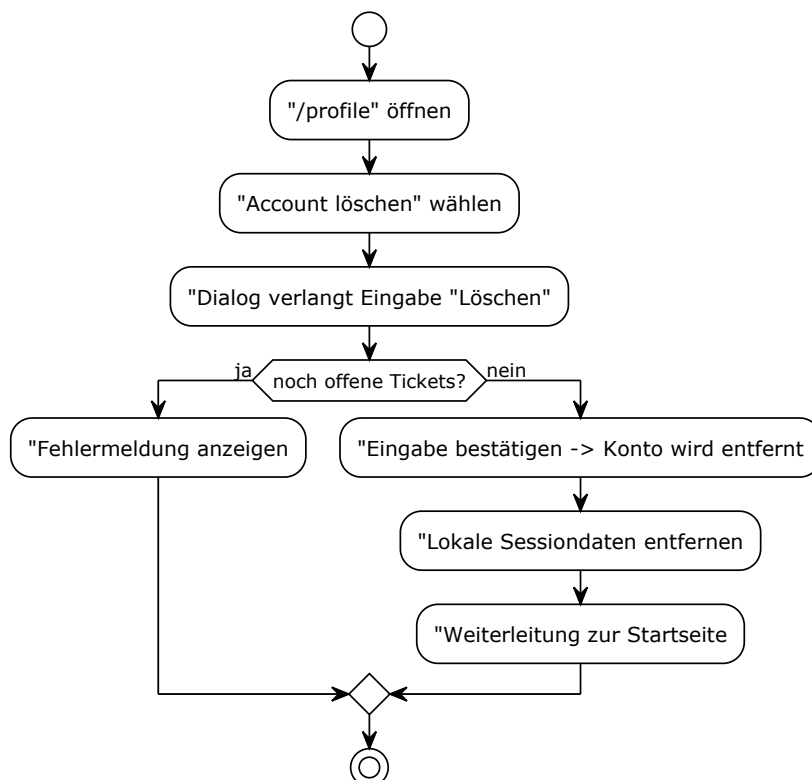
- Wenn keine Tickets verkauft wurden:
- Öffnen eines Bestätigungsdialogs
- Klick auf „Ja, löschen“
- Die Tour wird aus dem System entfernt
- Die Übersichtsliste wird automatisch neu geladen
- Wenn Tickets verkauft wurden:
- Die Löschung wird blockiert
- Anzeige eines Hinweises „Löschen nicht möglich“

Hinweise

- Bereits gebuchte oder verknüpfte Events/Tickets verhindern die Löschung aus Gründen der Datenkonsistenz
- Die Tour kann erst gelöscht werden, wenn alle zugehörigen Tickets storniert oder entfernt wurden

Benutzerkonto löschen

Konto löschen



Überblick

Dieser Prozess ermöglicht Nutzer*innen, ihr Konto eigenständig und dauerhaft zu löschen. Das System prüft dabei automatisch, ob noch offene oder nicht eingelöste Tickets vorhanden sind. Eine vollständige Löschung erfolgt nur bei erfüllten Voraussetzungen.

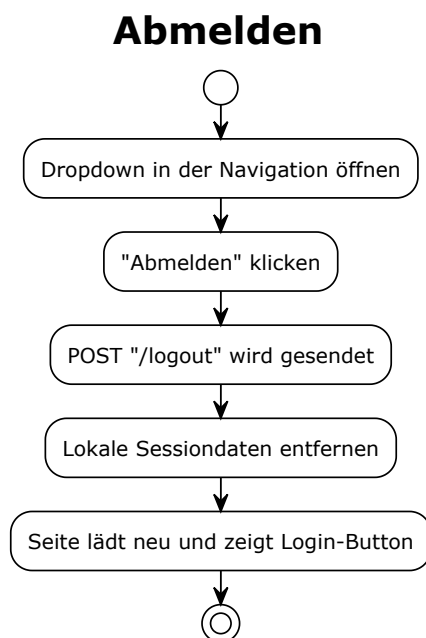
Ablauf

1. Aufrufen des eigenen Benutzerprofils unter `"/profile"`
2. Auswahl der Option „Account löschen“
3. Ein Dialog erscheint, der die Eingabe des Wortes „Löschen“ verlangt, um unbeabsichtigtes Entfernen zu vermeiden
4. Überprüfung durch das System, ob noch offene Tickets existieren

Entscheidung

- Wenn noch offene Tickets vorhanden sind:
- Abbruch der Löschung
- Anzeige einer Fehlermeldung, die auf die offenen Tickets hinweist
- Wenn keine offenen Tickets vorhanden sind:
- Bestätigung der Eingabe
- Das Konto wird dauerhaft entfernt
- Lokale Sitzungsdaten (Sessiondaten) werden gelöscht
- Weiterleitung zur Startseite

Abmelden



Überblick

Die Funktion "Abmelden" ermöglicht es Benutzern, ihre aktive Sitzung sicher zu beenden und sich vom System abzumelden. Dieser Prozess sorgt dafür, dass die Benutzerdaten geschützt bleiben,

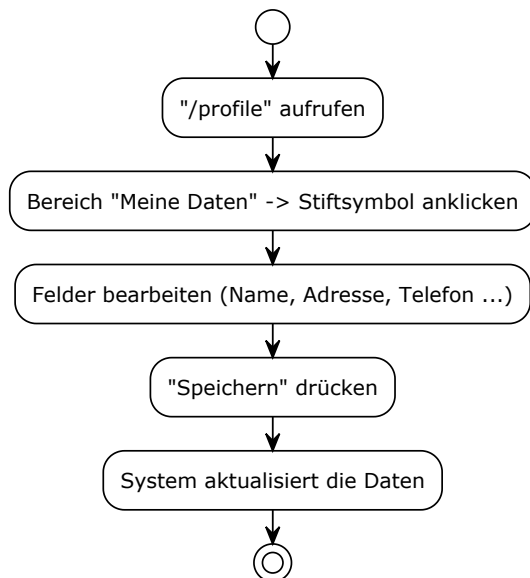
indem die lokale Session beendet wird und der Zugriff auf das Konto nur durch eine erneute Anmeldung möglich ist.

Ablauf

1. Dropdown in der Navigation öffnen:
 - 1.1. Der Prozess beginnt damit, dass der Benutzer ein Dropdown-Menü in der globalen Navigation öffnet, in dem die Abmeldeoption verfügbar ist.
2. "Abmelden" klicken:
 - 2.1. Innerhalb dieses Dropdowns klickt der Benutzer auf die Option "Abmelden".
3. POST "/logout" wird gesendet:
 - 3.1. Das System sendet daraufhin eine POST-Anfrage an den /logout-Endpunkt, um die serverseitige Sitzung zu beenden.
4. Lokale Sessiondaten entfernen:
 - 4.1. Parallel oder im Anschluss werden lokale Sessiondaten, wie Cookies oder Tokens, aus dem Browser des Benutzers entfernt. Dies ist ein kritischer Schritt für die Sicherheit.
5. Seite lädt neu und zeigt Login-Button:
 - 5.1. Nach erfolgreicher Abmeldung lädt die Seite neu und präsentiert dem Benutzer den Login-Button oder die Login-Maske, was anzeigt, dass die Sitzung beendet wurde und eine erneute Anmeldung erforderlich ist.

Profildaten aktualisieren

Profildaten bearbeiten



Überblick

Die Funktion "Profildaten bearbeiten" ermöglicht es Benutzern, ihre persönlichen Informationen wie Name, Adresse und Telefonnummer im System zu aktualisieren. Der Prozess ist darauf ausgelegt, eine einfache und direkte Bearbeitung der eigenen Daten zu gewährleisten.

Ablauf

1. Aufruf des Profilbereichs:

1.1. Der Prozess beginnt mit dem Aufrufen des persönlichen Profilbereichs, typischerweise unter dem Pfad /profile.

2. Initiierung der Bearbeitung:

2.1. Im Profilbereich wählt der Benutzer den Bereich "Meine Daten" aus und klickt auf das zugehörige Stiftsymbol, um den Bearbeitungsmodus zu aktivieren.

3. Bearbeitung der Felder:

3.1. Der Benutzer kann nun die gewünschten Felder wie Anrede, Vorname, Nachname, Firma, Straße und Hausnummer, PLZ, Stadt, Land, E-Mail, Geburtsdatum und Telefon bearbeiten oder aktualisieren.

4. Speichern der Änderungen:

4.1. Nachdem die Änderungen vorgenommen wurden, klickt der Benutzer auf die Schaltfläche "Speichern".

Systemprüfung und Aktualisierung

- System aktualisiert die Daten:
- Das System verarbeitet die eingegebenen Informationen und aktualisiert die Profildaten des Benutzers in der Datenbank.

Testkonzept

Zur Validierung der Ergebnisse aus der Applikation werden Tests durchgeführt, die sowohl Unit- als auch Integrationstestfälle umfassen. Geprüft werden hierbei die REST-Endpunkte des Backends, die Funktionsweise der React-Komponenten sowie komplette Nutzerflüsse mittels End-to-End-Tests.

User-Tests

User-Tests bezeichnen systematische Überprüfungen, bei denen reale Benutzer mit dem System interagieren, um dessen Benutzerfreundlichkeit, Funktionalität und Zuverlässigkeit zu bewerten. Bei diesen Tests werden typische Anwendungsfälle durchgespielt, um Probleme zu identifizieren, bevor sie in der Produktivumgebung auftreten.

In unserem Fall konzentrieren sich die User-Tests auf folgende Aspekte:

- Zugriffsberechtigungen und Rollenbeschränkungen
- Datenkonsistenz bei Löschoperationen
- Validierung von Eingaben und Geschäftsregeln
- Barrierefreiheit und Behindertenunterstützung

Die Tests simulieren reale Nutzungsszenarien und prüfen, ob das System wie erwartet reagiert, besonders in Grenzsituationen wie dem Versuch, mehr Tickets zu kaufen als verfügbar sind oder Daten zu löschen, die noch in Verwendung sind.

Als Normalnutzer wurde zunächst versucht, durch direkte Eingabe der URL „/admin/“ auf den Admin-Bereich zuzugreifen (TC01). Das System leitete jedoch konsequent auf die Anmeldeseite um. Anschließend folgte eine Reihe von Löschtests: Beim Versuch, eine Stadt zu entfernen, in der bereits ein Event existiert (TC02), erschien ein Popup mit dem Hinweis, dass das Löschen fehlschlug, weil Städte dieses Landes in einem Stadion verwendet werden. Gleiches geschah beim Löschen einer Venue mit bestehendem Event (TC03); hier meldete das System, die Venue könne nicht entfernt werden, da Tickets für Events existieren. Auch das Löschen eines Landes, in dem Events stattfinden (TC04), wurde mit einem entsprechenden Fehler-Popup verhindert, das erneut auf verwendete Städte verwies.

Testfall-ID	Testfall-Beschreibung	Aktion/Schritte	Tatsächliches Ergebnis
TC01	URL-Test: Zugriff auf Admin-Bereich als Normalnutzer	Als normaler Nutzer über URL '/admin/...' auf Admin-Seite navigieren	Umleitung auf Anmeldeseite
TC02	Löschtest: Stadt bei bestehendem Event in dieser Stadt	Versuch, eine Stadt zu löschen in der es ein Event gibt	Popup unterhalb der Darstellung, "Fehler beim Löschen da Städte dieses Landes in einem Stadion verwendet werden"
TC03	Löschtest: Venue bei bestehendem Event in dieser Venue	Versuch, eine Venue zu löschen in dem es ein Event gibt	Popup unterhalb der Darstellung, "Venue kann nicht gelöscht werden, da Tickets für Events existieren"
TC04	Löschtest: Land bei bestehendem Event in diesem Land	Versuch, ein Land zu löschen in dem es ein Event gibt	Popup unterhalb der Darstellung, "Fehler beim Löschen da Städte dieses Landes in einem Stadion verwendet werden"
TC05	Löschtest: Nutzerlöschen aus Usermaske bei bestehender Buchung eines Events	Versuch, einen Nutzer (aus User-Oberfläche) zu löschen bei einer bestehenden Buchung eines Events	Schrift unterhalb der Darstellung, "Sie können Ihr Konto nicht löschen, da sie Tickets für Veranstaltungen haben, die noch nicht stattgefunden haben."
TC06	Löschtest: Event mit vorhandenen Tickets	Versuch, ein Event zu löschen, für das bereits Tickets verkauft wurden	Nicht möglich. Keine Option zum Löschen von Events mit Tickets
TC07	Löschtest: Tour mit Events und Tickets	Versuch, eine Tour zu löschen, die Events mit verkauften Tickets enthält	Nicht möglich. Keine Option zum Löschen von Events mit Tickets
TC08	Löschtest: Artist mit Event	Versuch, einen Artist zu löschen	Artist sind im Ingesamten nicht Löschar.
TC09	Löschtest: Genre mit Event	Versuch, ein Genre zu löschen, für das bereits ein Ticket eines Events exestiert	Popup unterhalb der Darstellung, "Fehler beim Löschen des Genres. Möglicherweise ist dieses Genre mit einer Tour verbunden."
TC10	Löschtest: Sub-Genre mit Event	Versuch, ein Genre zu löschen	Popup unterhalb der Darstellung, "Fehler beim Löschen des Sub-Genres. Möglicherweise ist dieses Sub-Genre mit einer Tour verbunden."
TC11	Mehr Tickets von Usern bestellen als Tickets vorhanden.	Event erstellt, bei dem es nur ein Ticket gibt. Versucht für dieses Event mehr Tickets zu kaufen als vorhanden	Bei der Maximalen Ticketzahl (die in dem Fall durch die Anzahl der Tickets maximiert ist) graut sich der Kauf-Button aus. Damit ist das tatsächliche Kaufen eine Tickets nicht mehr möglich.
TC12	Löschtest: User löscht eigenen account obwohl ein Offener Schwerbehindertenantrag besteht	Versuch, ein User zu Löschen der einen offenen Schwerbehinderten Antrag hat	Antrag wird aus Liste der offenen Anträge gelöscht.
TC13	Für ein Event was angelegt wird, keine Sitzplätze anbieten (ob "normal" oder für Schwerbehinderte ist hier egal)	Versuch, ein Event in einer Vanue mit Behindertenbereich anzulegen und keine Tickets für Schwerbehinderte anzubieten sollte nicht erlaubt sein	Nicht erlaubt. Kategorien/Sitzplätze die hinzugefügt werden müssen einen Wert von >0 haben. Damit werden gesetzliche Vorgaben eingehalten.
TC14	Für ein Event mehrere Schwerbehindertentickets kaufen (ob Gratis begleitperson oder Sondertickets)	Versuch, mehrere Tickets zu kaufen, die mit einem Status eines Schwerbehinderten erlangt werden.	Nicht möglich. "In den Warenkorb"-Button nicht möglich anzuklicken. Dieser wird Ausgegraut.

Der Versuch, einen Nutzer direkt über die Benutzeroberfläche zu löschen, obwohl noch Buchungen für zukünftige Veranstaltungen bestehen (TC05), scheiterte ebenfalls: Unterhalb der Darstellung erschien der Text, das Konto könne nicht gelöscht werden, solange Tickets für noch nicht stattgefunden Events vorhanden sind. Ein Event mit bereits verkauften Tickets zu löschen (TC06) war gar nicht erst vorgesehen – die Option existierte schlicht nicht; dasselbe galt für das Entfernen einer Tour, die Events mit verkauften Tickets enthält (TC07). Artists ließen sich generell nicht löschen (TC08). Beim Versuch, ein Genre zu entfernen, für das bereits Tickets eines Events existieren (TC09), zeigte das System ein Popup, das einen Fehler beim Löschen meldete und als mögliche Ursache eine Tour-Verknüpfung nannte. Identisch verhielt es sich beim Sub-Genre (TC10): Auch hier verhinderte ein Fehlerhinweis das Löschen und verwies auf eine mögliche Verbindung zu einer Tour.

Ein weiterer Test zielte darauf ab, mehr Tickets zu bestellen, als tatsächlich vorhanden waren (TC11). Wurde ein Event mit nur einem Ticket angelegt und der Nutzer versuchte, mehr zu kaufen, graute sich der Kauf-Button bei Erreichen der maximalen Ticketanzahl aus, sodass ein weiterer Erwerb unmöglich war. Interessant war auch der Fall, in dem ein Nutzer sein Konto löschen wollte, obwohl noch ein offener Schwerbehindertenantrag bestand (TC12): Statt den Löschvorgang zu blockieren, entfernte das System den Antrag aus der Liste offener Anträge.

Beim Anlegen eines Events in einer Venue mit Behindertenbereich wurde geprüft, ob sich Sitzplätze – egal ob regulär oder für Schwerbehinderte – komplett weglassen lassen (TC13). Das System erlaubte dies nicht: Kategorien bzw. Sitzplätze müssen einen Wert größer null haben, womit gesetzliche Vorgaben eingehalten werden. Schließlich wurde getestet, ob mehrere

Schwerbehindertentickets (etwa Sondertickets oder Gratis-Begleitpersonen) gleichzeitig gekauft werden können (TC14). Auch das war nicht möglich: Der „In den Warenkorb“-Button blieb ausgegraut und ließ sich nicht anklicken.

Backend-Tests

Die automatisierten Backend-Tests nutzen Jest und prüfen alle API-Routen auf korrekte Antwortcodes sowie auf die Validierung der Eingabedaten. Hierbei wird eine isolierte Testdatenbank verwendet, damit produktive Daten nicht beeinflusst werden. Die Tests können mittels `npm run test:backend` ausgeführt werden.

Die Test-Suite deckt folgende Testfälle ab:

Test-Kategorie	Testfall	Beschreibung
Nutzer-API	Registrierung	Prüft, ob neue Nutzer angelegt werden können und ob Validierungsfehler korrekt zurückgegeben werden
	Login	Validiert den Login-Prozess mit korrekten und falschen Anmeldedaten
	Profildaten	Testet Abruf und Aktualisierung von Nutzerprofilen
Event-API	Event-Listing	Überprüft, ob Events korrekt nach verschiedenen Kriterien gefiltert werden
	Kategorie-Zuordnung	Validiert die korrekte Zuweisung von Kategorien zu Events
	Verfügbarkeit	Testet, ob die Verfügbarkeitslogik korrekt funktioniert
Bestellprozess	Warenkorb	Prüft das Hinzufügen und Entfernen von Artikeln
	Checkout	Validiert den kompletten Checkout-Prozess
	Bestellung	Testet die Erstellung einer Bestellung
Nachteilsausgleich	Antragsvalidierung	Überprüft die Validierung von Disability-Anträgen
	Genehmigungsprozess	Testet den Workflow zur Genehmigung/Ablehnung

Die Tests simulieren reale API-Aufrufe und validieren sowohl Erfolgs- als auch Fehlerszenarien. Jeder Testfall enthält mehrere Assertions, die spezifische Aspekte der API-Antwort überprüfen, wie HTTP-Status, Datenstruktur und Geschäftslogik.

Die Testumgebung wird vor jedem Testlauf zurückgesetzt, um Seiteneffekte zu vermeiden. Für die Simulation von authentifizierten Anfragen werden temporäre Sessions erstellt, die nach Abschluss des Tests automatisch bereinigt werden.

Nächste Schritte

Das Grundgerüst funktioniert lokal stabil: Registrierung, Login, Rollenverwaltung und der Bestellprozess sind lauffähig. Das Projekt ist in Frontend und Backend klar getrennt und setzt auf PostgreSQL als persistente Datenbasis. Dennoch gibt es mehrere Bereiche, die vor einer Produktivschaltung verbessert werden sollten:

Technische Optimierungen

1. Session-Management verbessern

- Implementierung eines persistenten Session-Stores (Redis/PostgreSQL)
- Aktuelle MemoryStore-Lösung verliert Sessions bei Neustarts

2. Datenbankverbindungen optimieren

- Connection-Pooling-Parameter anpassen
- Prepared Statements konsequenter einsetzen

3. Frontend-Performance steigern

- Bundle-Größe durch Code-Splitting reduzieren
- Lazy Loading für große Komponenten implementieren

Sicherheitsmaßnahmen

1. Eingabevalidierung erweitern

- Schema-Validierung auf allen Endpunkten
- Sanitization aller Benutzereingaben

2. Dateisicherheit verbessern

- MIME-Type-Validierung bei Uploads verstärken
- Größenbegrenzung für hochgeladene Bilder

Deployment-Verbesserungen

1. CI/CD-Pipeline

- Automatisierte Tests einrichten
- Docker-Compose für Entwicklung und Staging

2. Monitoring einführen

- Health-Checks und Logging implementieren
- Performance-Metriken erfassen

Die Session-Verwaltung und die Eingabevalidierung sollten prioritär angegangen werden, da sie die Stabilität und Sicherheit der Anwendung direkt beeinflussen.

Anhang

Datenbank-Modell

Tabelle	Zweck / wichtige Spalten	Abhängigkeiten
countries	Länder mit ISO-Code.	–
cities	Städte innerhalb eines Landes.	<code>countries</code> via <code>country_id</code>
user_roles	Definiert Rollen und zugehörige Rechte.	–
users	Registrierte Personen samt Adresse und optionalen Disability-Angaben.	<code>user_roles</code> via <code>role</code>
artists	Angaben zu Künstlern.	–
genres / subgenres	Klassifikation von Touren.	<code>genres</code> via <code>genre_id</code>
tours	Übergeordnete Tour einer Reihe von Events.	–
tour_artists	Zuordnung Künstler ↔ Tour.	<code>artists</code> , <code>tours</code>
tour_genres tour_subgenres	/ Genre-Verknüpfungen einer Tour.	<code>tours</code> , <code>genres</code> / <code>subgenres</code>
venues	Veranstaltungsorte.	<code>cities</code> via <code>city_id</code>
areas	Sitzplatz- bzw. Zugänglichkeitsbereiche.	–
venue_areas	Bereiche innerhalb eines Venues.	<code>venues</code> , <code>areas</code>
events	Konkrete Veranstaltungstermine.	<code>tours</code> , <code>venues</code>
event_categories	Ticket-Kategorien (Preis, Support für Behinderte).	<code>events</code> via <code>event_id</code>
event_supporting_acts	Support-Acts eines Events.	<code>events</code> , <code>artists</code>
event_venue_areas	Zuordnung Event ↔ Bereich mit Kapazität.	<code>events</code> , <code>venue_areas</code> , <code>event_categories</code>
images	Speicherung von hochgeladenen Bildern.	Beliebige Entität über <code>entity_type</code> / <code>entity_id</code>
carts	Aktiver Warenkorb eines Nutzers.	<code>users</code>
cart_items	Einzelne Artikel im Warenkorb.	<code>carts</code> , <code>events</code> , <code>event_categories</code>

Tabelle	Zweck / wichtige Spalten	Abhängigkeiten
checkouts	Zwischenschritt zwischen Warenkorb und Bestellung.	users
checkout_items	Positionen des Checkouts inkl. Preis.	checkouts , events , event_categories
payment_options shipping_options	/ Stammdaten für Zahlungs- und Versandarten.	-
orders	Abgeschlossene Bestellungen.	users , payment_options
tickets	Konkrete Ticketdatensätze.	orders , event_categories
order_tickets	Relation zwischen Order und Ticket (1-n).	orders , tickets
disability_marks	Mögliche Merkmale auf Behindertenausweisen.	areas via area_id
user_disability_marks	Zuordnung User ↔ Marks.	users , disability_marks

Frameworks

Paket	Version
@types/node	22.15.21
@types/react	19.1.5
concurrently	^9.1.2

Libraries/Packages

Die folgenden Tabellen listen alle im Projekt genutzten Pakete samt Version auf. Die Angaben stammen aus `package.json`.

Paket	Version
@testing-library/dom	^10.4.0
@testing-library/jest-dom	^6.6.3
@testing-library/react	^16.3.0
@testing-library/user-event	^13.5.0
bcrypt	^6.0.0
cors	^2.8.5
express	^5.1.0
express-session	^1.18.1
multer	^2.0.0
next	^15.3.2
nodemon	^3.1.10
opossum	^5.0.1
pg	^8.16.0
prop-types	^15.8.1
react	^19.1.0
react-dom	^19.1.0
react-router-dom	^7.6.1
react-scripts	^0.0.0
react-toastify	^11.0.5
toastify	^2.0.1
uuid	^11.1.0
web-vitals	^2.1.4